ALOUANI Fadi

BTS SIO 1
SLAM
12/11/26

[REACT] TP2 : gestion d'état, formulaires et API

Sommaire

Introduction....1
Partie 1 : Réorganisation du projet et état centralisé....2
Partie 2 : Suppression et changement de statut....3
Partie 3 : Formulaire contrôlé d'ajout....4
Partie 4 : Recherche, filtre et tri....5
Partie 5 : Appel à une API avec useEffect....6
5.2 Charger les tickets....8
Test de l'API....8
Partie 6 : Navigation avec React Router....9
Partie 7 : Finitions....9
Conclusion....9

Introduction

Dans ce TP, j'ai poursuivi le développement de mon application de gestion de tickets avec React. L'objectif était d'améliorer mon projet en ajoutant de nouvelles fonctionnalités, comme la création, la modification et la suppression de tickets, ainsi que la recherche et le filtrage. J'ai également travaillé sur la gestion des formulaires, l'utilisation d'une API et la navigation entre plusieurs pages avec React Router

ALOUANI Fadi

Partie 1 : Réorganisation du projet et état centralisé



```jsx
import TicketList from "./components/TicketList";

function App() {
  const [traites, setTraites] = useState(0);

  // Liste des tickets stockée dans un state
  const [tickets, setTickets] = useState([
    {
      id: 1,
      titre: "Imprimante en panne",
      auteur: "M. Durand",
      priorite: "Haute",
      statut: "Ouvert"
    },
    {
      id: 2,
      titre: "Accès VPN refusé",
      auteur: "Mme Martin",
      priorite: "Moyenne",
      statut: "En cours"
    },
    {
      id: 3,
      titre: "Écran qui clignote",
      auteur: "M. Petit",
      priorite: "Basse",
      statut: "Résolu"
    }
  ]);

  return (
    <div>
      <Header />

      <p>Bienvenue sur l'application WebDev Solutions.</p>

      <TicketList tickets={tickets} />

      <hr />

      <h2>Compteur de tickets traités</h2>

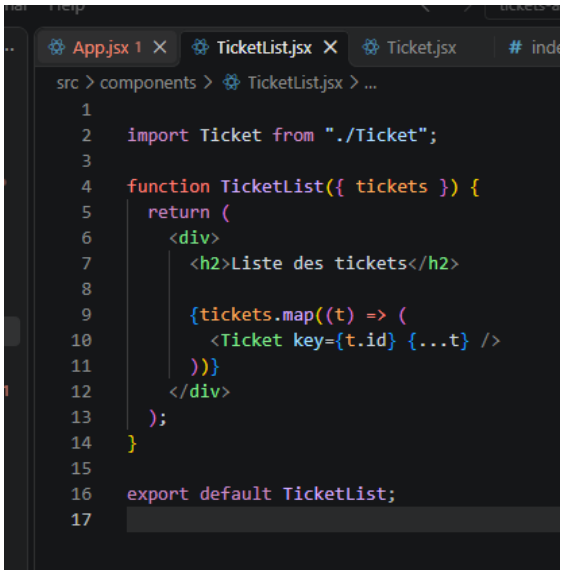
      <p>Tickets traités : {traites}</p>

      <button onClick={() => setTraites(traites + 1)}>
        Marquer un ticket comme traité
      </button>

      <button onClick={() => setTraites(0)}>
        Remettre à zéro
      </button>

      <button onClick={() => setTraites(Math.max(0, traites - 1))}>
        Annuler
```

Créez `TicketList.jsx` qui reçoit `tickets` en props et affiche la liste avec `map`.



App.jsx 1 | TicketList.jsx | Ticket.jsx

src > components > TicketList.jsx > ...

```jsx
import Ticket from "./Ticket";

function TicketList({ tickets }) {
  return (
    <div>
      <h2>Liste des tickets</h2>

      {tickets.map((t) => (
        <Ticket key={t.id} {...t} />
      ))}
    </div>
  );
}

export default TicketList;
```

ALOUANI Fadi

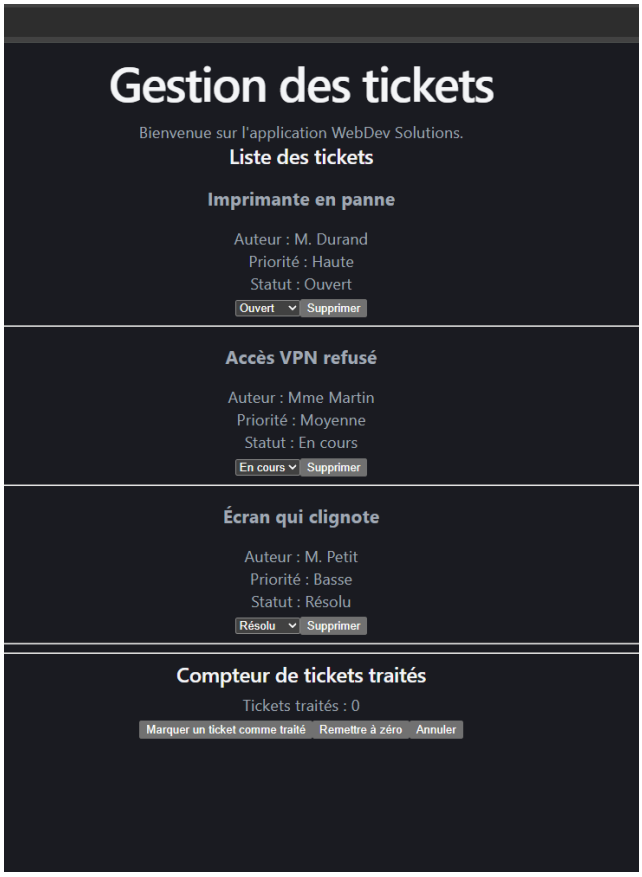
Question 1. Pourquoi stocke-t-on l'état dans le composant parent plutôt que dans TicketList ?

Pour centraliser les données et permettre à plusieurs composants d'accéder à la même liste de tickets grâce aux props.

Question 2. Qu'appelle-t-on le flux de données unidirectionnel en React ?

C'est le principe selon lequel les données circulent du composant parent vers les composants enfants grâce aux props.

Partie 2 : Suppression et changement de statut



Question 3. Pourquoi ne faut-il jamais utiliser push, splice ou modifier directement un objet du state ?

Car cela modifie directement les données existantes. React risque alors de ne pas détecter correctement les changements. Il faut créer un nouveau tableau ou objet pour mettre à jour le state.

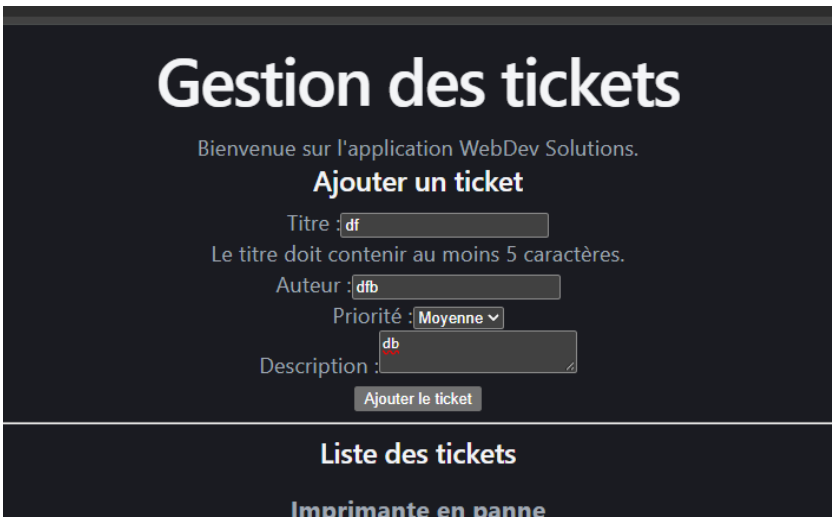
Question 4. Que fait l'opérateur de décomposition ...t ?

Il permet de copier toutes les propriétés d'un objet dans un nouvel objet. Ici, il conserve les informations du ticket et permet de modifier uniquement son statut.

Partie 3 : Formulaire contrôlé d'ajout

Créez `TicketForm.jsx` avec les champs : titre, auteur, priorité (liste déroulante), description (zone de texte).





Question 5. Quelle différence entre composant contrôlé et non contrôlé ?

Un composant contrôlé utilise le state de React pour gérer la valeur des champs. Un composant non contrôlé gère directement ses valeurs dans le DOM.

Question 6. À quoi sert e.preventDefault() ? Que se passe-t-il sans lui ?

Il empêche le comportement normal du formulaire, notamment le rechargement de la page lors de l'envoi. Sans lui, la page se recharge et les données peuvent être perdues.

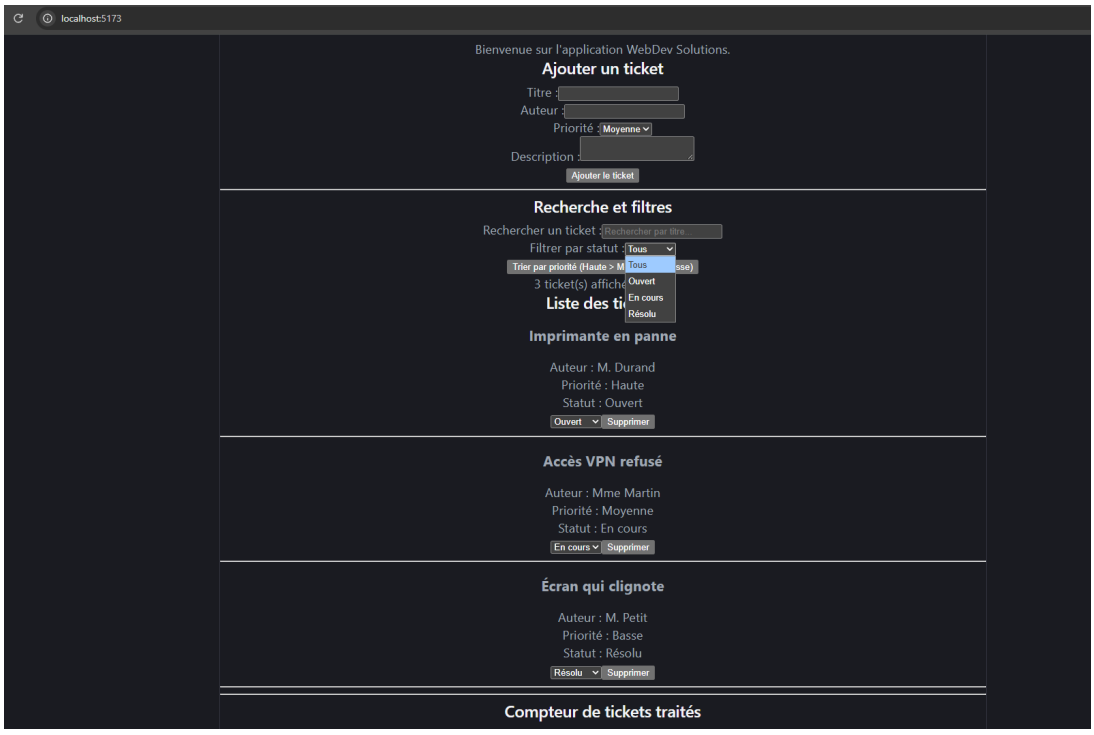
Question 7. Expliquez la syntaxe [e.target.name]: e.target.value.

Elle permet de modifier dynamiquement le champ correspondant à son attribut name avec la nouvelle valeur saisie

Partie 4 : Recherche, filtre et tri

Dans la page des tickets, ajoutez :

- un champ de **recherche** par titre (insensible à la casse) ;
- un filtre par **statut** (Tous / Ouvert / En cours / Résolu) ;
- un **tri** par priorité (Haute > Moyenne > Basse) via un bouton ou une liste.
- un compteur : « X ticket(s) affiché(s) sur Y ».



Question 8. Pourquoi vaut-il mieux calculer la liste filtrée plutôt que la stocker dans un second state ?

Cela évite de dupliquer les données et de devoir synchroniser plusieurs states. La liste filtrée est automatiquement recalculée lorsque les tickets ou les filtres changent.

Partie 5 : Appel à une API avec useEffect

Installez **json-server** et créez un fichier `db.json` à la racine du projet :

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\eleve\tickets-app> npm install -D json-server

added 42 packages, and audited 186 packages in 13s

45 packages are looking for funding
  run `npm fund` for details

4 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

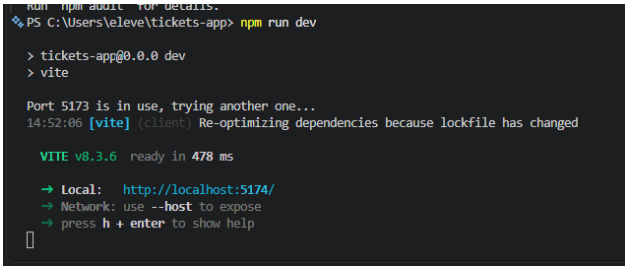
Run `npm audit` for details.
```

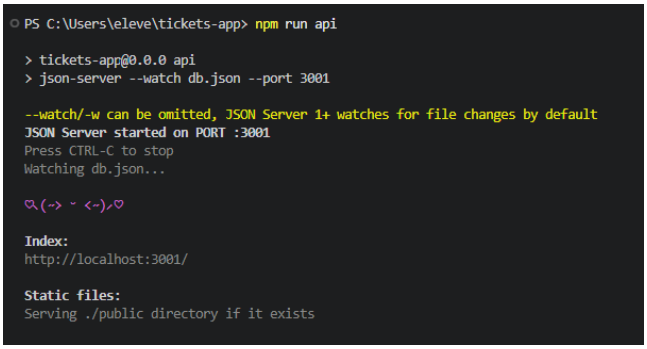
App.jsx | db.json | package.json | TicketList.jsx | Ticket.jsx

db.json > ...

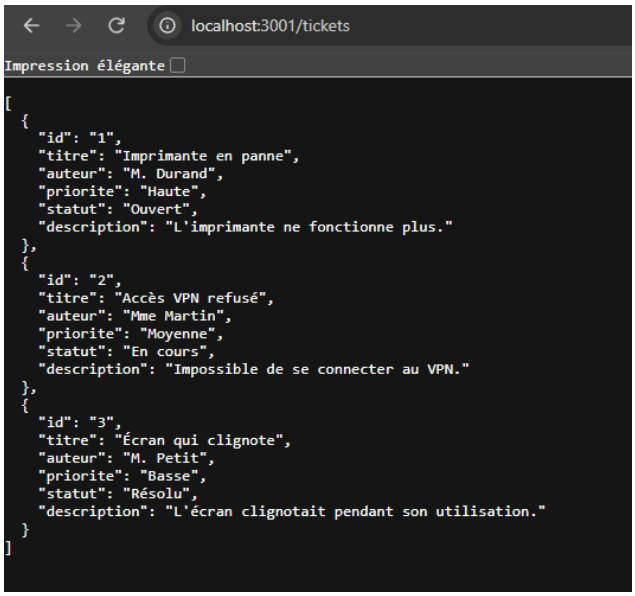
```json
{
  "tickets": [
    {
      "id": 1,
      "titre": "Imprimante en panne",
      "auteur": "M. Durand",
      "priorite": "Haute",
      "statut": "Ouvert",
      "description": "L'imprimante ne fonctionne plus."
    },
    {
      "id": 2,
      "titre": "Accès VPN refusé",
      "auteur": "Mme Martin",
      "priorite": "Moyenne",
      "statut": "En cours",
      "description": "Impossible de se connecter au VPN."
    },
    {
      "id": 3,
      "titre": "Écran qui clignote",
      "auteur": "M. Petit",
      "priorite": "Basse",
      "statut": "Résolu",
      "description": "L'écran clignotait pendant son utilisation."
    }
  ]
}
```

Ajoutez dans `package.json` le script `"api": "json-server --watch db.json --port 3001"` puis lancez-le dans un **second terminal** : `npm run api`.





Testez dans le navigateur : `http://localhost:3001/tickets`.



5.2 Charger les tickets

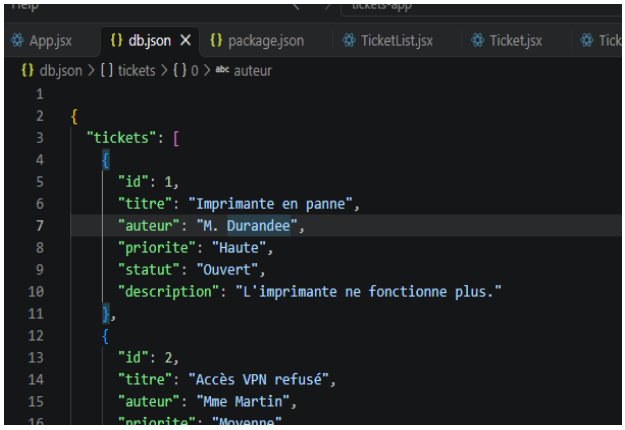
```
useEffect(() => {
  async function charger() {
    try {
      const reponse = await fetch("http://localhost:3001/tickets");

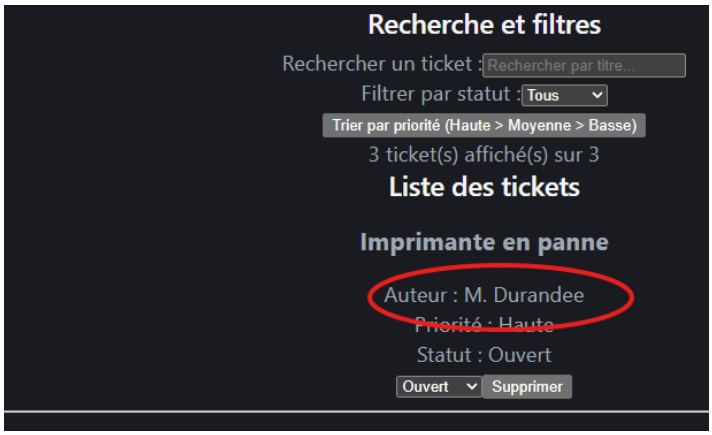
      if (!reponse.ok) {
        throw new Error("Erreur " + reponse.status);
      }

      const donnees = await reponse.json();
      setTickets(donnees);
    } catch (e) {
      setErreur(e.message);
    } finally {
      setChargement(false);
    }
  }

  charger();
}, []);
```

Test de l'API





Question 9. Que signifie le tableau de dépendances [] de useEffect ? Que se passe-t-il s'il est omis ?

Le tableau vide signifie que useEffect s'exécute au montage du composant. S'il est omis, useEffect

s'exécute après chaque rendu, ce qui peut provoquer des appels répétés à l'API.

Question 10. Citez les quatre verbes HTTP principaux et leur correspondance CRUD.

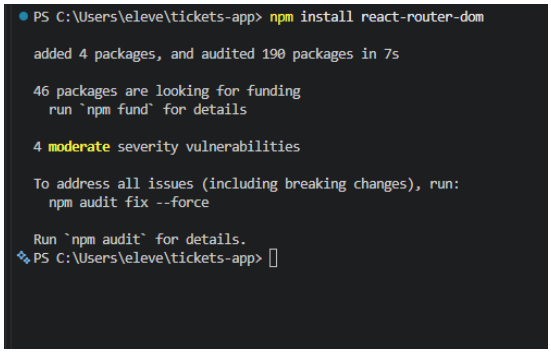
- GET : lire (Read).
- POST : créer (Create).
- PUT/PATCH : modifier (Update).
- DELETE : supprimer (Delete).

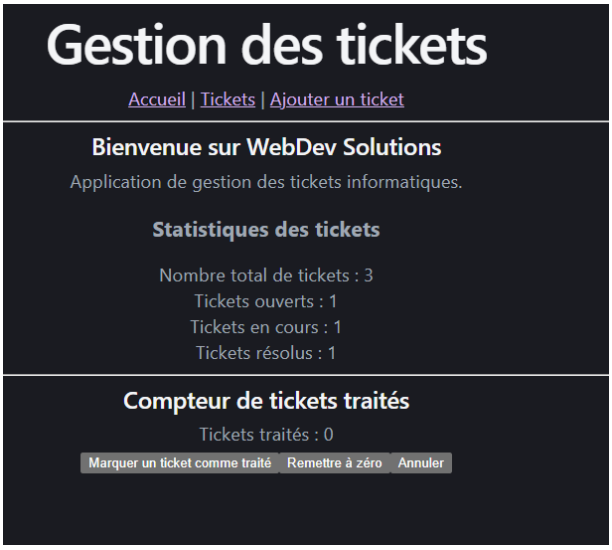
Question 11. Pourquoi useEffect ne peut-il pas recevoir directement une fonction async ?

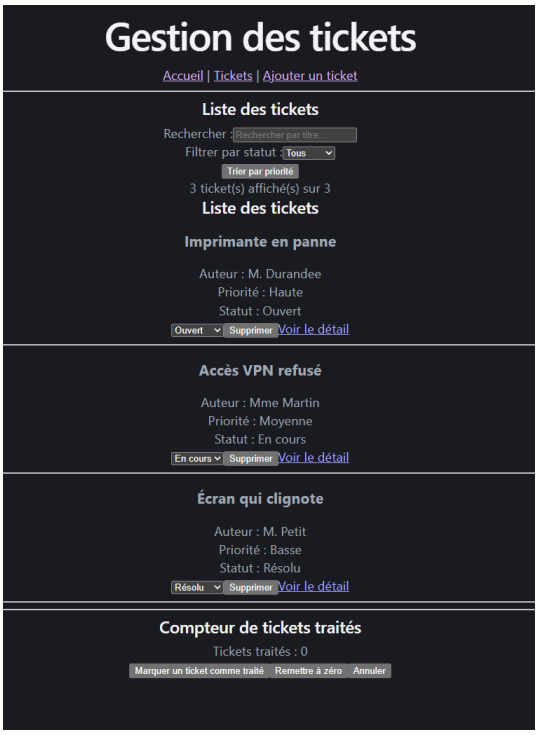
Parce qu'une fonction async retourne une promesse, alors que useEffect attend une fonction de nettoyage ou rien. Il faut donc créer une fonction async à l'intérieur de useEffect et l'appeler.

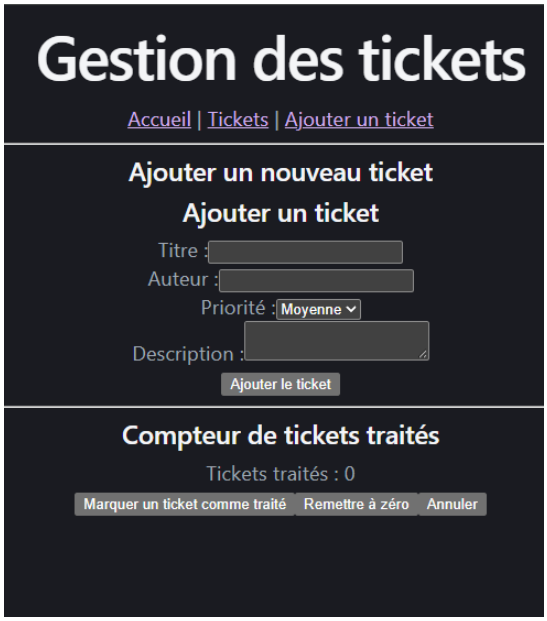
Partie 6 : Navigation avec React Router

Dans cette partie, j'ai mis en place la navigation avec React Router. J'ai créé plusieurs pages pour l'accueil, la liste des tickets, l'ajout et le détail d'un ticket. J'ai également ajouté un menu de navigation et une page 404. Les différentes pages fonctionnent correctement.







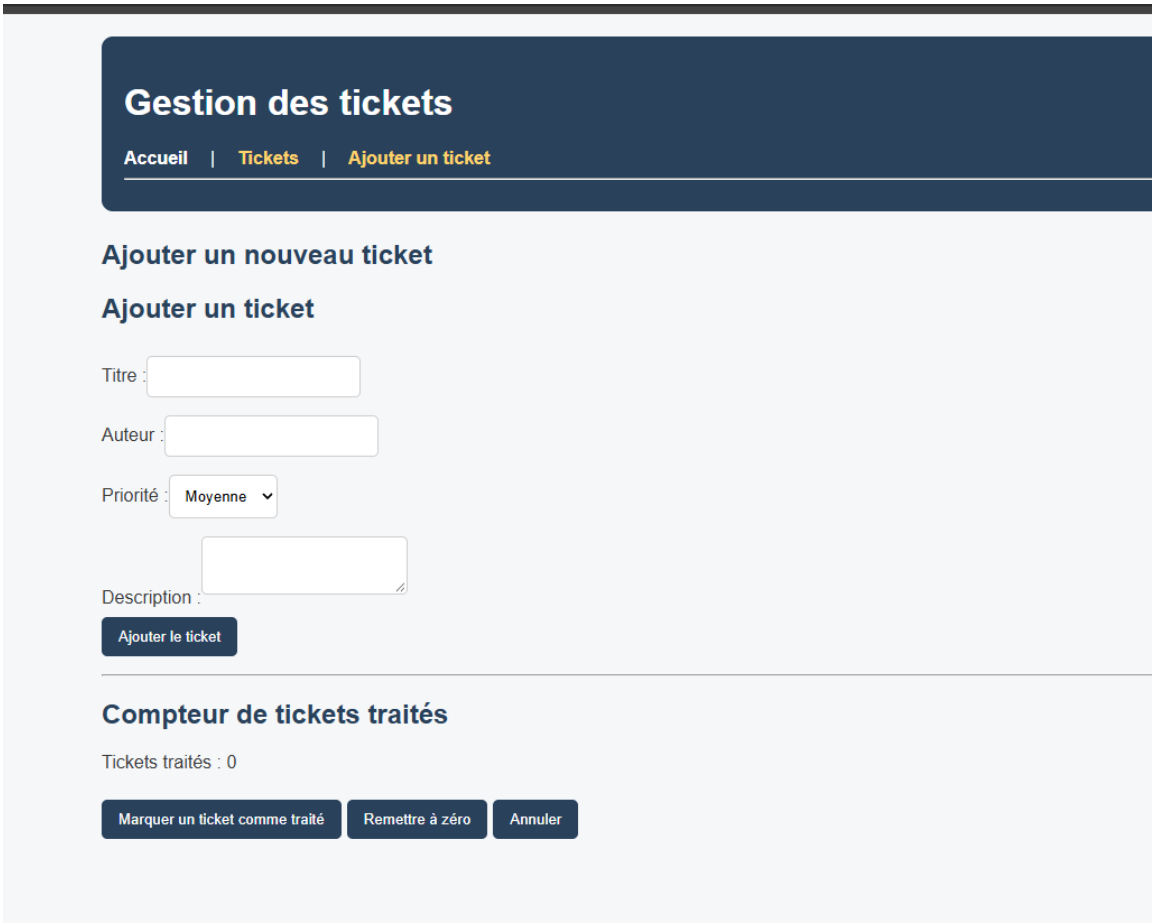


Question 12 – Compte rendu

Quelle différence entre <a href> et <Link> ? Quel est l'intérêt pour une SPA ?
<a href> recharge entièrement la page lors de la navigation, tandis que <Link> permet de changer de page sans rechargement. Cela rend la navigation plus rapide et fluide dans une SPA.

Partie 7 : Finitions

Accueille :



Conclusion

Pour conclure, ce TP m'a permis d'approfondir mes connaissances en React et de mieux comprendre la gestion des états, les formulaires contrôlés et les appels à une API. J'ai également appris à organiser mon projet avec des composants réutilisables et à mettre en place une navigation entre différentes pages. Ce travail m'a permis de rendre mon application plus complète et fonctionnelle, tout en améliorant mes compétences en développement web.